

AROGYA LINK

Detailed Implementation Plan & Technology Stack

Combined Architecture, Stack Rationale and Build-Level Detail per Phase

SIH 2026 SUBMISSION

Every phase pairs its implementation tasks with the exact technology, endpoints and tables that carry it — no stack decision floats free of the workflow it supports.

Contents

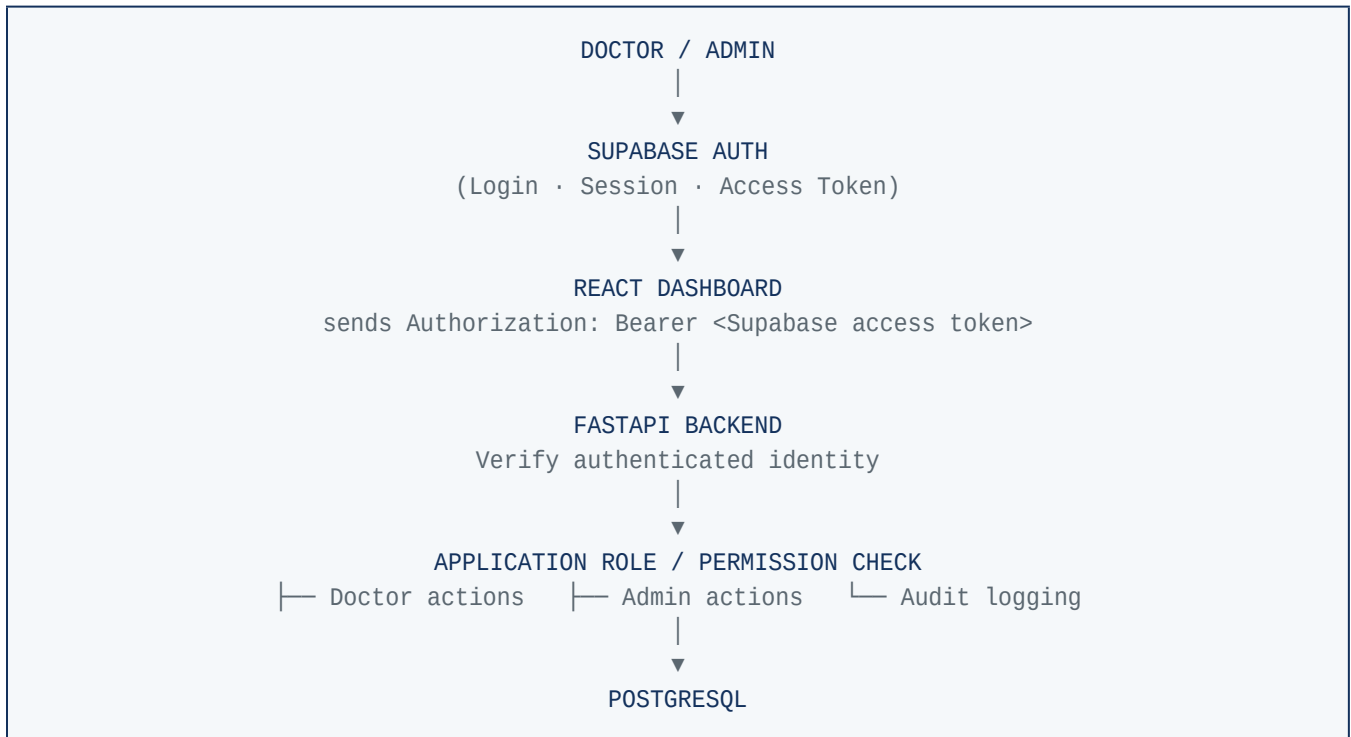
01 · Architecture Change Summary
02 · Final Technology Stack (Reference)
03 · Recommended Role Model
Phase 0 — Project Foundation
Phase 1 — Supabase Authentication Setup
Phase 2 — Staff Roles and Authorization
Phase 3 — Patient Session and Consent
Phase 4 — Adaptive Clinical Intake
Phase 5 — Deterministic Red-Flag Engine
Phase 6 — Doctor Dashboard
Phase 7 — Real-Time Escalation
Phase 8 — Document Upload and OCR
Phase 9 — Voice Input
Phase 10 — Gemini Clinical Summary
Phase 11 — Contradiction Detection
Phase 12 — Audit Logging
Phase 13 — Offline-First Kiosk
Phase 14 — History, Export, Testing and SIH Demo
Updated API and Data Flow
Recommended Build Order

01 · Architecture Change Summary

WHAT CHANGES

Custom password hashing, custom login endpoints, self-issued JWTs, and custom refresh-token handling are removed. Supabase Auth manages staff authentication and sessions. FastAPI remains responsible for backend business logic and authorization checks.

Staff Authentication Flow



02 · Final Technology Stack (Reference)

This table is the master reference. Each phase later in this document repeats only the rows relevant to it, alongside the concrete implementation tasks that use them — so the stack is never presented apart from the work it does.

Layer	Technology	Purpose
Patient Frontend	React + Vite	Patient/caregiver kiosk
Doctor Frontend	React + Vite	Staff dashboard
UI	Tailwind CSS or CSS	Responsive interface
Authentication	Supabase Auth	Doctor/Admin sign-in, sessions, token lifecycle
Frontend Auth SDK	supabase-js	Login, logout and session handling
Authorization	Supabase roles/custom claims + FastAPI checks	Restrict actions by role
Backend	FastAPI + Python	Main APIs and business logic
Identity Verification	Supabase-issued access token verification	Protect staff routes
Database	PostgreSQL	Clinical/application records and audit data
ORM	SQLAlchemy	Database models and queries
Validation	Pydantic	Request and AI-output validation
AI	Google Gemini API	Structured extraction and summaries
Speech	Bhashini / AI4Bharat	Primary multilingual STT
Speech Fallback	Whisper	Fallback transcription
OCR	PaddleOCR	Medical document extraction
Question Engine	Python + questions.json	Adaptive intake
Safety	Python + red_flags.json	Deterministic red-flag decisions
Real-Time	FastAPI WebSockets	Live escalation alerts
Offline Data	IndexedDB + idb	Queued kiosk answers
Offline Cache	Service Worker + Workbox	Offline assets and rules
Audit	PostgreSQL audit_log	Sensitive action history
Interoperability	FHIR-compatible JSON	Structured export

03 · Recommended Role Model

Role	Authentication	Main Permissions
Admin	Supabase Auth	Manage authorized staff/system settings; view permitted audit information.
Doctor	Supabase Auth	Review authorized records, red flags, summaries; edit and approve.
Patient / Caregiver	No staff login required	Use kiosk through encounter/session flow and consent.

Authorization is layered beneath authentication: a valid Supabase session only proves who the user is. Every sensitive backend endpoint independently checks what that authenticated identity is permitted to do — this is enforced in Phase 2, not assumed from the frontend.

Phase 0 — Project Foundation

Objective

Create a reproducible full-stack baseline that both frontends and the backend can be built on top of, with the database wired up before any feature work starts.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Patient Frontend	React + Vite	Scaffold for the kiosk app
Doctor Frontend	React + Vite	Scaffold for the staff dashboard
UI	Tailwind CSS or CSS	Shared design tokens and layout primitives
Backend	FastAPI + Python	App factory, health check, error handling
Database	PostgreSQL	Local dev instance via Docker Compose
ORM	SQLAlchemy	Engine, session dependency, migrations via Alembic
Validation	Pydantic	Settings management and base schemas

Frontend

- Scaffold two independent Vite apps — `apps/patient-kiosk` and `apps/doctor-dashboard` — each on React 18, so the kiosk and staff bundles never share risk of a shipped auth bug.
- Configure Tailwind via PostCSS with a small shared preset (colors, spacing, font scale) so both apps look consistent without duplicating a full design system yet.
- Add `.env.local` in each app for `VITE_API_BASE_URL`, `VITE_SUPABASE_URL`, `VITE_SUPABASE_PUBLISHABLE_KEY` — values are wired in Phase 1, but the env contract is fixed now.
- Set up React Router v6 in both apps; stub a shared `Layout` and a placeholder `ProtectedLayout` in the dashboard (auth logic added in Phase 1).

Backend

- Use an app-factory pattern — `create_app()` returning a configured `FastAPI` instance — so the same app can be imported by both `uvicorn` and the test suite.
- Add `CORSMiddleware` restricted to the kiosk and dashboard origins only (no wildcard `*` once staff auth exists).
- Implement `GET /health` returning `{status, db, version}`, where `db` reflects a live `SELECT 1` against PostgreSQL.
- Configure the SQLAlchemy engine and a `get_db()` dependency yielding a scoped session per request.
- Add a Pydantic v2 `Settings(BaseSettings)` class reading `DATABASE_URL`, `SUPABASE_URL`, `SUPABASE_JWT_AUD`, `GEMINI_API_KEY`, and related secrets from `.env` — never hard-coded.
- Add centralized exception handlers for `HTTPException`, `RequestValidationError`, and unhandled 500s, all returning one consistent JSON error envelope `{error_code, message, request_id}`.

Database

- Initialize Alembic for migrations (`alembic init migrations`) with an autogenerate-friendly `env.py` pointed at the SQLAlchemy metadata.

- Add `docker-compose.yml` with a local PostgreSQL service (with a persisted volume) so every contributor runs an identical dev database.
- Create migration `0001_init` establishing the database/schema/extensions baseline (e.g. `uuid-oss` or `pgcrypto` for UUID primary keys used throughout later phases).

DELIVERABLE

Both frontends communicate with FastAPI and database health is verified.

Phase 1 — Supabase Authentication Setup

Objective

Replace custom authentication with Supabase Auth for Doctor/Admin access, so FastAPI never touches a raw password.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Authentication	Supabase Auth	Issues and manages staff sessions and access tokens
Frontend Auth SDK	supabase-js	Sign-in, sign-out, session restore in the dashboard
Identity Verification	Supabase-issued access token verification	FastAPI validates every staff request

Supabase Setup

- Create the Supabase project and record the Project URL and publishable (anon) key — the publishable key is safe client-side; the service role key is never shipped to any frontend.
- Under Auth → Providers, enable Email; disable public self-sign-up since this is a staff-only system — accounts are provisioned, not registered.
- Create the initial authorized staff users directly in the Supabase dashboard (or via the Admin API using the service role key from a trusted server context only).
- Under Auth → URL Configuration, add the dev origin (e.g. `http://localhost:5173`) and the deployed dashboard URL as allowed redirect/site URLs.

Doctor Frontend

- `npm install @supabase/supabase-js` and create `lib/supabaseClient.ts` via `createClient(VITE_SUPABASE_URL, VITE_SUPABASE_PUBLISHABLE_KEY)`.
- Implement sign-in with `supabase.auth.signInWithPassword({email, password})` and sign-out with `supabase.auth.signOut()`.
- Subscribe to `supabase.auth.onAuthStateChange` and mirror the session into a shared `AuthContext` so every component re-renders consistently on login/logout.
- On app load, call `supabase.auth.getSession()` to restore a persisted session before rendering protected routes.

- Wrap dashboard routes in a `ProtectedRoute` that checks for a session and redirects unauthenticated users to `/login`.
- Attach `Authorization: Bearer <session.access_token>` to every FastAPI request via a fetch/axios interceptor, refreshing through `supabase.auth.refreshSession()` on expiry.

FastAPI

- Delete the old `/auth/register`, `/auth/login`, `/auth/refresh` endpoints and any password-hashing utilities — they no longer exist in this architecture.
- Implement `get_current_user()` as a FastAPI dependency: extract the Bearer token, verify its signature against Supabase's JWKS endpoint (or the project JWT secret), and check `aud=authenticated` plus `exp`.
- Cache the fetched JWKS keys in memory with a periodic refresh, so token verification doesn't cost a network round-trip on every request.
- On success, build a lightweight `AuthUser(id, email)` from the verified token claims; role is resolved separately in Phase 2.
- Raise `401 Unauthorized` for missing, malformed, or expired tokens before any handler logic runs — identity is checked at the dependency layer, not inside business logic.

DELIVERABLE

A Doctor/Admin can sign in through Supabase and access protected FastAPI routes.

Phase 2 — Staff Roles and Authorization

Objective

Separate authentication (who is this?) from authorization (what can they do?) so a valid session alone never grants access to a restricted action.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Authorization	Supabase roles/custom claims + FastAPI checks	Role source of truth and per-endpoint enforcement
Backend	FastAPI + Python	<code>`require_role`</code> dependencies
Database	PostgreSQL	<code>`staff_profiles`</code> table

Implementation

- Define exactly two staff roles for this phase: `doctor` and `admin`.
- Store role assignment server-side in a `staff_profiles` table keyed by the Supabase user id, rather than trusting a client-editable `user_metadata` field.
- Extend `get_current_user()` so it joins the verified Supabase user id to `staff_profiles` and attaches the resolved role to the request-scoped user object.

- Implement `require_role("doctor") / require_role("admin")` as dependencies that wrap `get_current_user()` and raise `403 Forbidden` on a role mismatch; expose them as `require_doctor` and `require_admin`.
- Declare `Depends(require_doctor)` or `Depends(require_admin)` explicitly on every sensitive endpoint added from Phase 3 onward — permission checks live on the backend route, never inferred from a hidden frontend button.
- Reserve a small set of admin-only endpoints for staff management: inviting/deactivating accounts and viewing the full audit log (built out in Phase 12).

Database Tables Introduced

TABLE `staff_profiles`

Column	Type	Notes
<code>id</code>	uuid (PK)	Matches the Supabase Auth user id
<code>full_name</code>	text	
<code>role</code>	text	'doctor' 'admin'
<code>active</code>	boolean	Deactivated staff are rejected at the dependency layer
<code>created_at</code>	timestampz	

DELIVERABLE

A valid signed-in user cannot perform actions outside their assigned role.

Phase 3 — Patient Session and Consent

Objective

Create a traceable anonymous/semi-anonymous patient encounter flow so every later action (intake, documents, red flags) has somewhere to attach to.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Backend	FastAPI + Python	Session/consent endpoints, no staff auth required here
Database	PostgreSQL	<code>`patients`</code> , <code>`encounters`</code> , <code>`consent`</code> tables
Validation	Pydantic	Consent payload and encounter-state schemas

Implementation

- Collect only the patient details required for the encounter (name/age band, optional phone) — kept minimal since this is a public kiosk flow, not a staff login.

- Display the consent text and require an explicit affirmative action before anything clinical is recorded; persist the consent event, not just a boolean.
- Generate a server-issued encounter/session id (UUID) the moment consent is recorded; every kiosk screen after this point references that id.
- Reject any intake, document upload, or answer submission where the referenced encounter has no recorded consent — enforced server-side, not just hidden in the UI.

Key API Endpoints

Method	Endpoint	Purpose
POST	/session	Create a patient + encounter record, return encounter_id
POST	/consent	Record consent for a given encounter_id
GET	/session/{encounter_id}	Fetch encounter status (used to resume/validate a kiosk session)

Database Tables Introduced

TABLE `patients`

Column	Type	Notes
id	uuid (PK)	
display_name	text	Optional
age_band	text	
phone_optional	text	Nullable
created_at	timestampz	

TABLE `encounters`

Column	Type	Notes
id	uuid (PK)	The encounter/session id used throughout the app
patient_id	uuid (FK → patients)	
status	text	'open' 'closed'
started_at	timestampz	
closed_at	timestampz	Nullable

TABLE `consent`

Column	Type	Notes
id	uuid (PK)	
encounter_id	uuid (FK → encounters)	
consent_given	boolean	
consent_text_version	text	So consent copy changes stay auditable
signed_at	timestampz	

DELIVERABLE

Every intake action is linked to a valid encounter.

Phase 4 — Adaptive Clinical Intake

Objective

Build the complete non-AI clinical question workflow first, so the core intake works with zero dependency on Gemini or voice.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Question Engine	Python + questions.json	Deterministic next-question selection
Patient Frontend	React + Vite	Complaint selection and adaptive question UI
Database	PostgreSQL	`answers` table

Frontend

- Build the complaint-selection screen as the entry point into intake, feeding the chosen complaint into `question_engine`.
- Render the adaptive question screen driven entirely by what the backend returns next — the frontend holds no branching logic of its own.
- Use large touch targets and inline validation suited to a kiosk (no keyboard-first assumptions).
- Show a progress indicator derived from the question engine's estimate of remaining depth, not a hard-coded step count.

Backend

- `question_engine.py` loads `questions.json` once at startup and keeps it in memory — no per-request file reads.
- Implement `select_next(question_bank, answered_so_far)` as a pure function, independent of FastAPI, so it can be unit-tested directly against fixtures.
- Persist each answer as it's submitted (not batched at the end), so a mid-intake disconnect never loses prior answers.

Key API Endpoints

Method	Endpoint	Purpose
GET	<code>/intake/next-question</code>	Return the next question for a given encounter_id and answer history
POST	<code>/intake/answer</code>	Persist an answer; triggers red-flag evaluation (Phase 5)

Database Tables Introduced

TABLE `answers`

Column	Type	Notes
<code>id</code>	uuid (PK)	
<code>encounter_id</code>	uuid (FK → encounters)	
<code>question_id</code>	text	Key from questions.json
<code>value</code>	jsonb	Supports scalar or structured answers
<code>answered_at</code>	timestampz	

DELIVERABLE

A patient completes the entire touch-only intake without Gemini or voice.

Phase 5 — Deterministic Red-Flag Engine

Objective

Implement reproducible safety screening that never depends on a model call, so the same input always produces the same safety result.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Safety	Python + <code>red_flags.json</code>	Explicit, versioned rule evaluation
Database	PostgreSQL	<code>`red_flags`</code> table

Implementation

- Define `red_flags.json` as a list of explicit rules, e.g. `{rule_id, when: {question, equals|gte|lte}, severity}` — no free-text conditions, so every rule is independently reviewable.
- Evaluate the relevant subset of rules in pure Python immediately after each answer is persisted (triggered from `POST /intake/answer`).
- Return, for every triggered rule, the `severity`, the `rule_id`, and the specific answers that satisfied it (the evidence) — never a bare true/false.
- Persist every result (triggered or not, depending on audit needs) in the `red_flags` table so the safety history of an encounter is fully reconstructable.

Database Tables Introduced

TABLE `red_flags`

Column	Type	Notes
<code>id</code>	uuid (PK)	

Column	Type	Notes
encounter_id	uuid (FK → encounters)	
rule_id	text	e.g. RF-CHEST-PAIN-01
severity	text	'low' 'medium' 'high'
evidence	jsonb	The specific answers that triggered it
triggered_at	timestamptz	

IMPORTANT RULE

Gemini must not independently determine emergency priority. Red-flag severity comes only from this deterministic engine, and Phase 10's summaries are generated after — and reference — this result, never the reverse.

DELIVERABLE

The same input always produces the same rule-based safety result.

Phase 6 — Doctor Dashboard

Objective

Provide a unified staff review interface over the deterministic workflow, before any AI-generated content exists to review.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Doctor Frontend	React + Vite	Queue, detail view, review/approve UI
Authentication	Supabase Auth	Session gating every dashboard screen
Backend	FastAPI + Python	Read/query endpoints for queue and detail data

Implementation

- Build the patient queue as the dashboard landing screen, sorted with open escalations and high-severity red flags surfaced first.
- Build the patient detail view assembling: clinical answers, triggered red flags, uploaded documents, AI summary status, and detected contradictions in one screen.
- Implement the review/edit/approve workflow as explicit, auditable actions — an approval is a recorded event (Phase 12), not just a UI state change.
- Gate every screen behind the `ProtectedRoute` and role checks from Phases 1–2; a doctor and an admin see different available actions on the same data.

Key API Endpoints

Method	Endpoint	Purpose
GET	/patients/queue	List open encounters, prioritized by escalation/severity
GET	/patients/{encounter_id}	Full detail bundle: answers, red flags, documents, summary, contradictions

DELIVERABLE

A doctor can review the deterministic patient workflow before AI is integrated.

Phase 7 — Real-Time Escalation

Objective

Immediately notify connected staff the moment a deterministic priority rule fires, without requiring a dashboard refresh.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Real-Time	FastAPI WebSockets	Push alerts to every connected staff client
Database	PostgreSQL	`escalations` table
Doctor Frontend	React + Vite	WebSocket client and live alert UI

Backend

- Create an escalation record whenever a **high** (or configured threshold) severity red flag is triggered.
- Implement **EscalationManager** as a connection registry mapping authenticated staff sessions to open WebSocket connections.
- Expose **/ws/staff-alerts** as an authenticated WebSocket endpoint (token passed on connect, validated the same way as REST requests).
- Broadcast a minimal alert payload — **{encounter_id, rule_id, severity, created_at}** — not the full clinical record, keeping the socket payload small and fast.

Frontend

- Open a WebSocket client on dashboard load, authenticated with the current Supabase access token.
- Show a live alert banner/toast on new escalation events, with a direct link into that patient's detail view.
- Implement automatic reconnect with backoff when the connection drops, so a brief network blip doesn't silently stop alerts.

Key API Endpoints

Method	Endpoint	Purpose
WS	/ws/staff-alerts	Authenticated live channel broadcasting new escalations

Database Tables Introduced

TABLE `escalations`

Column	Type	Notes
<code>id</code>	uuid (PK)	
<code>encounter_id</code>	uuid (FK → encounters)	
<code>rule_id</code>	text	
<code>status</code>	text	'open' 'acknowledged' 'resolved'
<code>acknowledged_by</code>	uuid (FK → staff_profiles)	Nullable
<code>acknowledged_at</code>	timestampz	Nullable
<code>created_at</code>	timestampz	

DELIVERABLE

Critical patient input creates a persistent escalation and a live staff alert.

Phase 8 — Document Upload and OCR

Objective

Convert uploaded medical documents into reviewable structured text, verified as working before Gemini ever touches the output.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
OCR	PaddleOCR	Extracts text from uploaded medical documents
Backend	FastAPI + Python	Upload validation and OCR orchestration
Database	PostgreSQL	`documents` table

Implementation

- Validate upload type (image/PDF only) and enforce a size limit before anything is written to storage.
- Store file metadata immediately on upload; store the file itself on disk or object storage referenced by `storage_path`.
- Run PaddleOCR asynchronously (or as a background task) so the upload request itself returns quickly with a `pending` status.
- Persist the raw extracted text alongside an `ocr_status` (`pending` / `done` / `failed`) so the dashboard can show progress honestly.
- Display both the original document and the extracted text side by side in the dashboard for doctor verification — OCR output is never treated as ground truth on its own.

Key API Endpoints

Method	Endpoint	Purpose
POST	/documents/upload	Upload a document for an encounter; returns document_id
GET	/documents/{document_id}	Fetch metadata, OCR status, and extracted text

Database Tables Introduced

TABLE documents

Column	Type	Notes
id	uuid (PK)	
encounter_id	uuid (FK → encounters)	
filename	text	
mime_type	text	
storage_path	text	
ocr_status	text	'pending' 'done' 'failed'
ocr_text	text	Nullable until OCR completes
uploaded_at	timestampz	

DELIVERABLE

OCR works independently before Gemini consumes the data.

Phase 9 — Voice Input

Objective

Support multilingual voice input while guaranteeing the touch-based flow from Phase 4 still works if voice fails.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Speech	Bhashini / AI4Bharat	Primary multilingual speech-to-text
Speech Fallback	Whisper	Used only when the primary STT is unavailable
Patient Frontend	React + Vite	Audio capture and transcript confirmation UI

Implementation

- Capture audio on the kiosk and send it to the backend rather than performing STT in-browser, keeping model access centralized.
- Call Bhashini/AI4Bharat as the configured primary STT for the patient's selected language.

- On timeout or error from the primary provider, fall back to Whisper transparently — the patient never sees which engine ran.
- Always show the resulting transcript for explicit confirmation before it's saved as an answer — voice output is treated as a draft, not a final value.

Key API Endpoints

Method	Endpoint	Purpose
POST	/voice/transcribe	Accepts an audio blob + encounter_id, returns a transcript for confirmation

DESIGN NOTE

Voice is strictly additive to Phase 4's touch flow. If both STT paths fail, the patient falls back to the same touch input already built and tested — intake never blocks on voice.

DELIVERABLE

Voice failure does not block the intake workflow.

Phase 10 — Gemini Clinical Summary

Objective

Generate structured clinical context only after the deterministic data flow (intake, red flags, OCR) is stable and independently working.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
AI	Google Gemini API	Structured extraction and clinical summary drafting
Validation	Pydantic	Validates Gemini's structured output before it is stored
Database	PostgreSQL	`summaries` table, versioned

Implementation

- Create `llm_service.py` as the single integration point with Gemini — no other module calls the API directly.
- Normalize answers, OCR text, and red-flag results into one application-controlled structured input; Gemini never receives raw, unvalidated user text alone.
- Request structured output (JSON mode / response schema) rather than free-form prose, and validate the response against a `ClinicalSummarySchema` Pydantic model.
- Store only output that passes validation; reject and log anything malformed or schema-invalid rather than silently repairing it.
- Preserve the Gemini-generated version and the doctor-edited/approved version as separate rows, never overwriting the original draft.

Failure Handling

- A Gemini timeout or API error must never block doctor access to the underlying deterministic intake data — the dashboard still shows answers, red flags, and documents with the summary marked as unavailable.
- Malformed or schema-invalid output is rejected and logged with enough context to reproduce the failure, without surfacing an unvalidated draft to the doctor.

Key API Endpoints

Method	Endpoint	Purpose
POST	/summaries/generate/{encounter_id}	Trigger Gemini summary generation for an encounter
POST	/summaries/{summary_id}/approve	Doctor approves/edits a generated summary

Database Tables Introduced

TABLE summaries

Column	Type	Notes
id	uuid (PK)	
encounter_id	uuid (FK → encounters)	
version	integer	
source	text	'gemini' 'doctor_edited'
content	jsonb	Validated structured summary
status	text	'draft' 'approved' 'failed'
created_at	timestampz	

DELIVERABLE
Doctor receives a validated Gemini-generated draft summary.

Phase 11 — Contradiction Detection

Objective

Surface inconsistencies between what the patient reported and what documents show, for human verification rather than automatic resolution.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Backend	FastAPI + Python	Explicit comparison rules between structured sources
Database	PostgreSQL	`contradictions` table

Implementation

- Compare structured patient answers against extracted document information field by field (e.g. reported medication vs. OCR-extracted prescription).
- Write explicit contradiction rules first — deterministic field comparisons — before considering any model-assisted matching.
- Store both source values plus a review status on each detected contradiction, never a single 'corrected' value.
- Show both sources side by side to the doctor; the system flags, it does not decide.

Database Tables Introduced

TABLE `contradictions`

Column	Type	Notes
<code>id</code>	uuid (PK)	
<code>encounter_id</code>	uuid (FK → encounters)	
<code>field</code>	text	e.g. 'current_medication'
<code>patient_reported_value</code>	text	
<code>document_value</code>	text	
<code>status</code>	text	'unresolved' 'reviewed'

DELIVERABLE

The system flags possible conflicts but does not automatically decide which source is correct.

Phase 12 — Audit Logging

Objective

Create accountability for every sensitive staff action, tied to a verified Supabase identity.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Audit	PostgreSQL <code>audit_log</code>	Append-only record of sensitive actions
Authentication	Supabase Auth	Source of the actor identity on every log row

Implementation

- Create `audit_log` capturing actor, action, resource type/id, timestamp, and relevant metadata for every write-sensitive operation.
- Record record views where appropriate (e.g. opening a patient detail view), plus edits, summary approvals, and escalation acknowledgements.
- Use the authenticated Supabase user id as the actor reference on every row — never a free-text name.

- Call a single `log_action()` helper from each sensitive endpoint rather than scattering ad-hoc logging, so the audit trail's shape stays consistent.

Database Tables Introduced

TABLE `audit_log`

Column	Type	Notes
<code>id</code>	uuid (PK)	
<code>actor_user_id</code>	uuid (FK → <code>staff_profiles</code>)	
<code>action</code>	text	e.g. 'summary.approve', 'escalation.acknowledge'
<code>resource_type</code>	text	e.g. 'encounter', 'summary'
<code>resource_id</code>	uuid	
<code>metadata</code>	jsonb	
<code>created_at</code>	timestampz	

DELIVERABLE

Important actions are traceable to an authenticated staff user.

Phase 13 — Offline-First Kiosk

Objective

Allow essential intake to continue through a temporary network failure, with the server re-asserting authority once connectivity returns.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Offline Data	IndexedDB + <code>idb</code>	Local queue for unsent answers
Offline Cache	Service Worker + Workbox	Caches app shell and rule/question configuration
Backend	FastAPI + Python	Idempotent sync endpoint

Frontend

- Queue unsent answers in IndexedDB via the `idb` helper library whenever a network write fails.
- Precache the app shell plus the `questions.json` and `red_flags.json` configuration with a Workbox-managed Service Worker.
- Clearly display offline status in the kiosk UI so staff and patients aren't left guessing whether an answer was saved.
- Run a local mirror of the required red-flag rules client-side, so an offline patient still gets an immediate on-device safety signal.

Backend

- Accept synchronized answers via an idempotent `/sync/answers` endpoint keyed on client-generated UUIDs, so retried syncs never duplicate rows.
- Handle duplicate/retry scenarios explicitly rather than relying on the client to sync exactly once.
- Re-run the server-side deterministic rules as authoritative after sync — the client-side red-flag mirror is a UX aid, not the system of record.

Key API Endpoints

Method	Endpoint	Purpose
POST	<code>/sync/answers</code>	Idempotent batch sync of queued offline answers

DELIVERABLE

Disconnecting the network mid-session does not immediately stop core intake.

Phase 14 — History, Export, Testing and SIH Demo

Objective

Prepare the system for a clean, reproducible end-to-end demonstration, backed by an explicit test matrix.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Interoperability	FHIR-compatible JSON	Structured export of an encounter
Backend	FastAPI + Python	pytest + httpx test suite

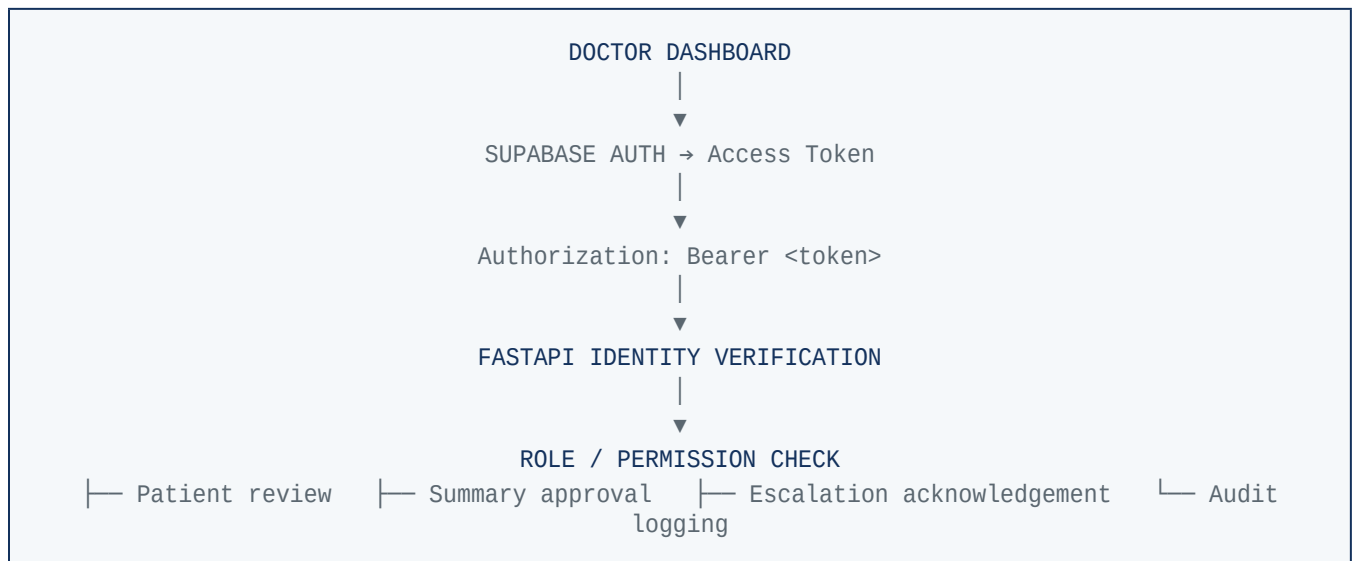
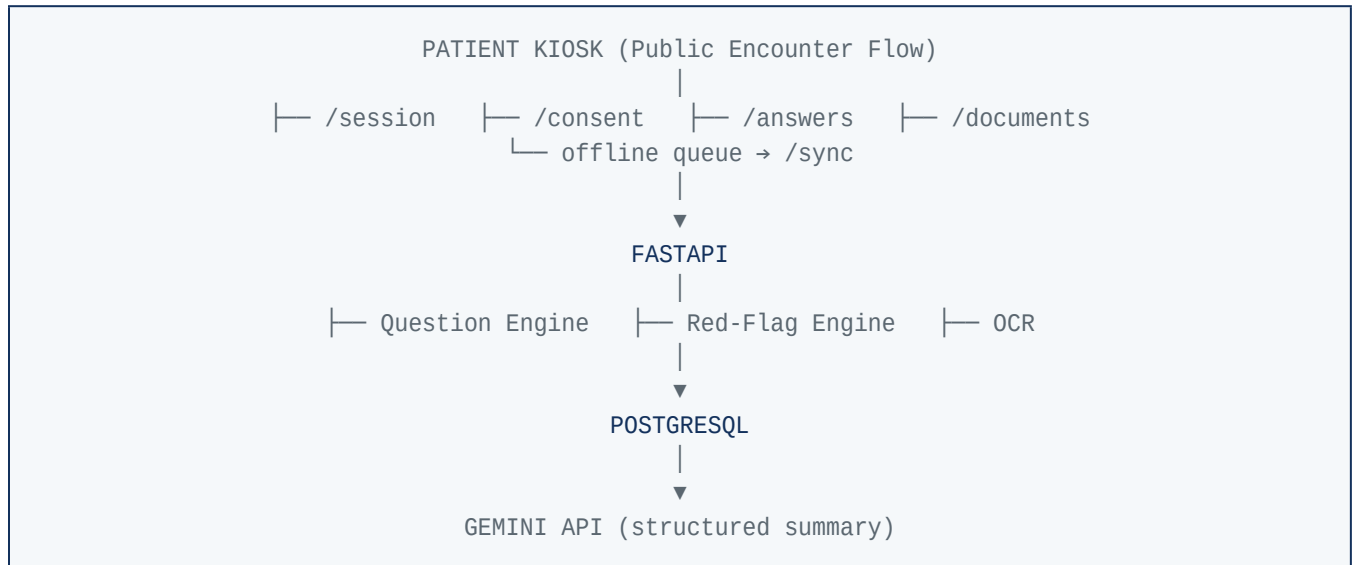
Implementation

- Build the patient/encounter timeline view, assembling intake, red flags, documents, summaries, and escalations in chronological order.
- Implement FHIR-compatible JSON export for a completed encounter.
- Write unit tests for the question engine and red-flag rules — pure-Python functions with no FastAPI dependency, so these are fast and deterministic.
- Write API tests (e.g. with `httpx.AsyncClient`) covering session/consent, intake, escalation, and summary endpoints.
- Write a WebSocket integration test asserting an escalation triggers a broadcast to a connected client.
- Write authentication/authorization tests confirming role checks reject out-of-role requests (Phase 2's core guarantee).
- Write an offline interruption and recovery test simulating a queued sync after disconnect.
- Write a Gemini-failure test confirming the dashboard remains usable when summary generation fails (Phase 10's failure handling).

DELIVERABLE

Clean end-to-end demo: intake → red flag → live alert → OCR → Gemini → doctor approval → audit.

Updated API and Data Flow



Recommended Build Order

1. Foundation and database connectivity
2. Supabase Auth and protected doctor dashboard
3. Roles and backend authorization
4. Patient session and consent
5. Adaptive intake
6. Deterministic red flags
7. Doctor dashboard review
8. Real-time escalation
9. Document upload and PaddleOCR
10. Voice input
11. Gemini structured summaries
12. Contradiction detection
13. Audit logging
14. Offline-first kiosk
15. History, FHIR export, integration tests and SIH demo

FINAL ARCHITECTURAL PRINCIPLE

Supabase Auth answers “Who is this staff user?” FastAPI answers “What is this authenticated user allowed to do?” Deterministic rules answer safety-critical red-flag questions. Gemini structures and summarizes clinical information. The doctor remains the final authority.

AROGYA LINK

Implementation Plan Addendum — Closing the SIH Pitch Gaps

WHY THIS ADDENDUM EXISTS

The SIH pitch deck (SIH26047) commits to five capabilities that the current 14-phase implementation plan does not yet build: ABHA/ABDM integration, AYUSH-specific assessment (Dashavidha Pariksha & Ahara-Vihara), a patient reminder system, smart follow-up (delta-only re-intake), and line-level traceability in the AI summary. This addendum adds five new phases, 15–19, following the same format as the original document, so they can be slotted into the existing plan without renumbering it.

Phase 15 — ABHA / ABDM Integration

Objective

Link every encounter to the patient's Ayushman Bharat Health Account so records are portable across facilities, fulfilling the "continuity of care" claim in the pitch — this goes beyond the generic FHIR export already planned in Phase 14.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
ABDM Gateway	ABDM Sandbox / M2 APIs	Health ID discovery, linking, consent-based data exchange
Backend	FastAPI + Python	ABDM client wrapper, callback handlers
Database	PostgreSQL	`abha_links` table
Interoperability	FHIR-compatible JSON	Payload format required by ABDM Health Information Exchange

Implementation

- Integrate the ABDM Health ID (ABHA) verification flow — mobile/Aadhaar-based OTP — at the patient-kiosk consent step (Phase 3), storing only the ABHA reference id, never the Aadhaar number itself.
- Register ArogyaLink as a Health Information User (HIU) / Health Information Provider (HIP) against the ABDM sandbox for the hackathon demo environment.
- On encounter close, push the FHIR bundle generated in Phase 14 to the patient's ABDM-linked Personal Health Record via the consent-manager callback, rather than only offering local export.
- Make ABHA linking optional but recommended: an encounter without a linked ABHA still completes normally, preserving the existing kiosk flow for walk-in/first-time patients.

TABLE *abha_links*

Column	Type	Notes
id	uuid (PK)	
patient_id	uuid (FK → patients)	

Column	Type	Notes
abha_id	text	ABDM health ID reference, not Aadhaar
consent_artifact_id	text	ABDM consent-manager reference
linked_at	timestampz	

Key API Endpoints

Method	Endpoint	Purpose
POST	/abha/link	Initiate ABHA verification for a patient and store the resulting reference
POST	/abha/push/{encounter_id}	Push the encounter's FHIR bundle to the linked ABDM PHR
DELIVERABLE A patient's encounter is discoverable and portable through their ABHA, not just exportable as a local file.		

Phase 16 — AYUSH Assessment Module

Objective

Add the Ayurvedic/AYUSH-specific intake the pitch names explicitly — Dashavidha Pariksha (the ten-fold Ayurvedic examination) and Ahara-Vihara (diet and lifestyle) assessment — as a parallel question set alongside the modern-medicine intake built in Phase 4.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Question Engine	Python + ayush_questions.json	Separate, versioned AYUSH question bank
Patient Frontend	React + Vite	AYUSH intake screens (Prakriti, Dashavidha Pariksha, Ahara-Vihara)
Database	PostgreSQL	`ayush_assessments` table

Implementation

- Model ayush_questions.json as its own question bank, reusing the Phase-4 question-engine contract (select_next) so no new branching logic is needed on the frontend.
- Cover the ten Dashavidha Pariksha parameters (Prakriti, Vikriti, Sara, Samhanana, Pramana, Satmya, Satva, Ahara Shakti, Vyayama Shakti, Vaya) as a structured, scorable question set rather than free text.

- Cover Ahara-Vihara as a diet/lifestyle questionnaire (meal timing, food qualities, sleep, activity) feeding into the same structured summary consumed in Phase 10.
- Let the institution/doctor select at encounter start whether the AYUSH module runs alongside or instead of the standard intake, so non-AYUSH facilities are unaffected.

TABLE *ayush_assessments*

Column	Type	Notes
id	uuid (PK)	
encounter_id	uuid (FK → encounters)	
prakriti	jsonb	Constitution assessment result
dashavidha_pariksha	jsonb	Structured ten-fold examination answers
ahara_vihara	jsonb	Diet and lifestyle answers
assessed_at	timestampz	

Key API Endpoints

Method	Endpoint	Purpose
GET	/ayush/next-question	Return the next AYUSH question for a given encounter
POST	/ayush/answer	Persist an AYUSH assessment answer
DELIVERABLE AYUSH institutions can capture Dashavidha Pariksha and Ahara-Vihara data as structured, doctor-reviewable output, not just modern-medicine intake.		

Phase 17 — Reminder System

Objective

Deliver the time-based medicine and follow-up reminders promised on the pitch deck, closing the loop after a doctor approves a summary.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Backend	FastAPI + Python	Reminder scheduling and dispatch logic
Scheduler	APScheduler / Celery beat	Time-based trigger for due reminders
Notification	SMS/WhatsApp gateway (e.g. MSG91) or push	Patient-facing delivery channel
Database	PostgreSQL	`reminders` table

Implementation

- Generate reminder rows automatically from an approved summary's medication list (Phase 10) and any doctor-set follow-up date — doctors can edit or add reminders manually.
- Run a scheduled job that scans for due reminders and dispatches them through the configured channel (SMS/WhatsApp/push), independent of whether the kiosk app is open.
- Record delivery status per reminder (sent/failed/acknowledged) so the doctor dashboard can show whether a patient's medication reminders are actually reaching them.
- Respect the phone_optional field from Phase 3 — patients who didn't share a phone number get on-screen/print reminders only, never a blocked flow.

TABLE reminders

Column	Type	Notes
id	uuid (PK)	
encounter_id	uuid (FK → encounters)	
type	text	'medicine' 'follow_up'
message	text	
scheduled_at	timestampz	
status	text	'pending' 'sent' 'failed' 'acknowledged'
created_at	timestampz	

Key API Endpoints

Method	Endpoint	Purpose
POST	/reminders	Create a reminder (manual or auto-generated) for an encounter
GET	/reminders/{encounter_id}	List reminders and delivery status for an encounter
DELIVERABLE Patients receive time-based medicine and follow-up		

Method	Endpoint	Purpose
reminders after their visit, with delivery status visible to staff.		

Phase 18 — Smart Follow-up

Objective

Use a patient's prior visit history so a returning patient answers only what changed, instead of repeating the full Phase-4 intake — the "Smart Follow-up" differentiator named in the pitch.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
Backend	FastAPI + Python	Prior-encounter diffing logic
Question Engine	Python + questions.json	Same engine, seeded with prior answers
Database	PostgreSQL	Reuses `encounters` / `answers`; adds `encounter_links`

Implementation

- At session creation (Phase 3), check whether the linked ABHA (Phase 15) or patient record has a prior closed encounter within a configurable lookback window.
- When a prior encounter exists, pre-populate the question engine's `answered_so_far` with the most recent value for each question instead of starting blank.
- Present pre-filled answers to the patient as "still true?" confirmations rather than silently carrying them forward — every carried-over answer is explicitly re-confirmed, never assumed.
- Only questions the patient changes, or new questions triggered by a changed answer, are written as new answer rows — unchanged confirmations are logged as confirmations, not duplicated.

TABLE encounter_links

Column	Type	Notes
id	uuid (PK)	
encounter_id	uuid (FK → encounters)	The current (follow-up) encounter
previous_encounter_id	uuid (FK → encounters)	The encounter it follows up from
linked_at	timestampz	

Key API Endpoints

Method	Endpoint	Purpose
GET	/intake/prior-context/{encounter_id}	Return the most recent prior encounter's answers, if any, for pre-fill
POST	/intake/confirm-unchanged	Record that a pre-filled answer was confirmed as still accurate
DELIVERABLE A returning patient's intake only asks about what's changed since their last visit, cutting repeat history-taking time.		

Phase 19 — Traceable AI Summary

Objective

Extend the explainability already built for red flags (Phase 5) to the entire Gemini-generated summary, so every line the doctor reads can be traced back to its voice, touch, or document source — the "Traceable AI Summary" claim on the pitch deck.

Tech Stack Used in This Phase

Component	Technology	Role in this phase
AI	Google Gemini API	Requested to emit source references alongside each summary line
Validation	Pydantic	Extends ClinicalSummarySchema with a provenance field per line
Database	PostgreSQL	Extends `summaries.content` structure; no new table required

Implementation

- Extend the ClinicalSummarySchema from Phase 10 so each summary line is an object {text, source_type, source_id} rather than a bare string — source_type is one of 'voice' | 'touch' | 'document'.
- When normalizing answers/OCR text/red-flag results into Gemini's structured input (Phase 10), tag each fact with its originating answer_id or document_id so Gemini can echo that reference back rather than inventing one.
- Reject and log any summary line returned without a resolvable source_id, the same fail-closed pattern already used for schema-invalid output in Phase 10 — an untraceable line is never shown to the doctor as if it were sourced.
- In the doctor dashboard (Phase 6), let clicking any summary line highlight the exact answer, transcript segment, or document region it came from.

Key API Endpoints

Method	Endpoint	Purpose
GET	/summaries/{summary_id}/trace/{line_id}	Return the source answer, transcript segment, or document reference for one summary line
DELIVERABLE Every line of the AI-generated summary is traceable to its originating voice, touch, or document input — not just the red-flag results.		

Updated Recommended Build Order

Insert the new phases relative to their dependencies, rather than strictly appending them at the end:

- ...(Phases 1–14 as in the original document)...
- 15. AYUSH assessment module — after Phase 4 (Adaptive Intake), since it reuses the question-engine contract
- 16. Smart follow-up — after Phase 3 (Patient Session) and Phase 4, since it depends on prior-encounter lookup
- 17. ABHA/ABDM integration — after Phase 14 (FHIR export), since it pushes the same FHIR bundle further
- 18. Reminder system — after Phase 10 (Gemini Summary), since reminders are generated from the approved summary
- 19. Traceable AI summary — folded into Phase 10 directly if time allows, otherwise last, since it only extends the existing summary schema

NOTE ON NUMBERING

The phases above are labelled 15–19 for reference in this addendum. When merging into the master document, they can be renumbered to sit adjacent to their dependency phase (e.g. AYUSH module as "Phase 4b") if a single continuous sequence is preferred for the SIH submission.